RePromptFuzz:
Improving LLM Generated Fuzz Drivers Through Iterative Prompting

12/3/2025

By Andrew Bush & Mitchell Windahl

Introduction

Fuzzing has long been a staple of the software analysis toolkit, enabling analysts to test thousands of possible inputs on a given piece of software. This approach is especially necessary when testing the security of libraries, which contain many functions that each need to be tested. However, simply using a developed fuzz tool and pointing it at a specific library will often not work. Libraries may require specialized input formats or unique data structures that have to be translated into from the original fuzz-generated input. Therefore, fuzz drivers must be employed to transform fuzz generated inputs into usable function calls. Once again, making these drivers is difficult due to the amount of background knowledge of the target library required to craft a quality driver. Noting these issues, Lyu et al created Prompfuzz, a fuzz driver generation tool that utilizes LLM models to generate high quality fuzz drivers. Nonetheless, Promptfuzz left some room for possible improvements. In this report, we demonstrate a proof of concept for incorporating iterative prompting to resolve syntactic errors within PromptFuzz to possibly improve performance and effectiveness.

Background

Related Work

Utilizing Large Language Models to aid in fuzzing and fuzz driver synthesis has been an increasingly researched topic over the past few years. During our initial research, we came across several works on the subject, each incorporating differing methods to refine and integrate LLMs into the fuzzing workflow. One of the more common methods we observed was the use of a

feedback system to correct error prone code. This method was seemingly first proposed by Zhang et al, whose research demonstrated that iteratively querying the LLM to fix problems in the code resulted in better performance and success rates compared to other methods. Indeed, we noticed that this method was present in many LLM related fuzzing projects, although there was rarely any explanation as to why this technique was chosen to be included. For instance, Amiet's fuzz-o-matic and researchers from google's security blog both developed AI powered fuzzing tools that included an iterative error correcting step (Liu et al 2023, 2024). Additionally, a 2024 literature review on the subject of LLMs in fuzzing technology specifically recommended iterative prompting as a means of addressing errors in fuzz driver synthesis (Jiang et al, 2024). For these reasons, we were particularly interested in why PromptFuzz neither chose to implement this feature nor explain their reason for its absence. While we hope the introduction of an iterative prompting mechanism will improve PromptFuzz, it is possible that the existence of negative impacts will serve as counter evidence for the notion of iterative prompting being more effective than PromptFuzz's brute force approach of discarding error-prone code.

PromptFuzz

Promptfuzz operates in a four step workflow to achieve maximum coverage and integration. PromptFuzz begins in the Instructive Program Generation phase by extracting function signatures and type definitions from the header files of a given C++ library. It then uses this information to generate a prompt which calls a given LLM to make programs that utilize those functions and types. PromptFuzz then executes the generated code and validates them based on runtime behavior, whilst eliminating erroneous code. PromptFuzz stores the valid programs in a seed bank, which mimics the typical greybox style fuzzing workflow. It uses this seedbank to

mutate its phase one prompts in a way which expands code coverage. PromptFuzz will then return to phase one with the mutated prompts and continue running until it either runs out of usable tokens or detects no expansion of code coverage within ten loops. Finally, PromptFuzz changes the constant values generated by the LLMs into variables which can take arbitrary values from a fuzzer. PromptFuzz then combines its valid seeds into a driver capable of receiving random byte input from a fuzzer (Lyu et al).

Our Contribution - RePromptFuzz

Given the presence of iterative LLM prompting for the purpose of correcting machine generated code in several other LLM powered fuzztools, we theorized that PromptFuzz could benefit from the increased performance such a system allegedly provides. To that end, we created a fork of PromptFuzz with the goal of inserting such a mechanism into the error checking phase of the workflow. We have dubbed this project RePromptFuzz, as to reference the implementation of the iterative loop.

In PromptFuzz's original implementation, code sanitization is handled by the sanitize.rs file. The file utilizes clang, LLVM, libfuzzer and AddressSantizer to ensure that syntax, links, execution, fuzz implementation, and coverage are correct. The file runs these checks in order, returning the first error it encounters. The sanitize.rs file is utilized by harness.rs, which actually parses the programs into the sanitization check function. Below is the pseudocode of the use of check in harness.rs.

```
Rust
BEGIN main
    // Parse configuration from CLI
    config ← parse_config()

    // Initialize test environment using project path
    initialize_test_environment(config.project)

    project_path ← copy_of(config.project)

    // Check which command was requested
    IF config.command is "check" THEN

        // Run the "check" function, which attempts sanitization checks
        result ← run_check(project_path, program_argument)

        // The check() function returns either:
        //   - Some(error) if sanitization failed
        //   - None if sanitization succeeded

        IF result contains an error THEN
            // Convert the error to JSON
            error_json ← serialize_error_to_json(result.error)

            // Print error to stderr
            print_error(error_json)

            // Exit with failure code 41
            EXIT with code 41
        ELSE
            // No errors found
            EXIT with success code 0
        ENDIF
    ENDIF
END main
```

We chose this as the location to implement our error correction loop. Our rationale for this decision was that we could attempt to fix a program that returns an error before an error code 41 is returned in harness.rs, allowing broken code to be repaired before its discarded. We wrote a file ‘llm_repairs.rs’ which handles quiring the LLM to fix the generation. The module prompts

the same designated LLM to repair the erroneous code and provides the error as additional context. When a result is received, the program extracts the generated code and returns it to harness.rs. This program is then checked as usual. If the generated program is also erroneous, the cycle repeats for a user-specified number of times, defaulting to 3 if never specified. Code that fails to be fixed within that limit is discarded as normal. Below is the pseudocode of the modified harness.rs file which calls upon llm_repairs.rs:

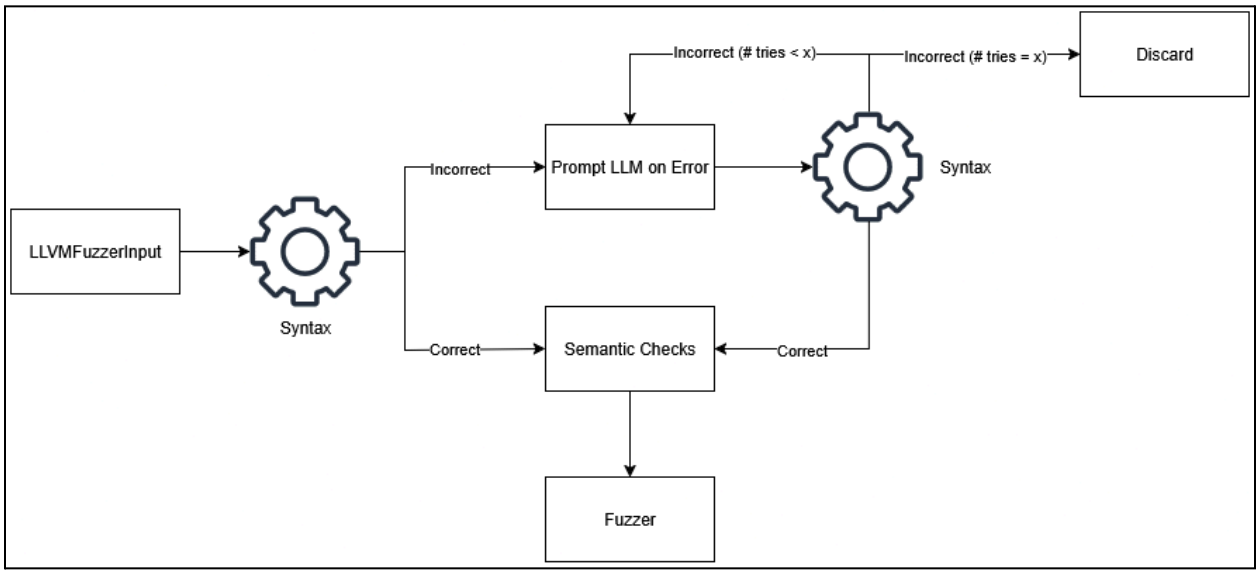
Figure 1: LLM Repair Pseudocode

```
Rust
BEGIN main
    config ← parse_config()

    initialize_test_environment(config.project)

    project_path ← copy_of(config.project)

    IF config.command is "check" THEN

        // Use provided max_retries if given, otherwise default to 3
        retries_allowed ← config.max_retries OR 3

        // Run the check() function, which:
        //   - attempts sanitization
        //   - if sanitization fails, calls the repair module
        //   - retries sanitization up to retries_allowed times
        //   - finally returns:
        //         Some(error) if all repairs fail
        //         None if sanitization eventually succeeds
        result ← run_check(project_path, program_argument, retries_allowed)

        IF result contains an error THEN
            error_json ← serialize_error_to_json(result.error)
            print_error(error_json)
            EXIT with code 41
        ELSE
            EXIT with success code 0
        ENDIF

    ENDIF
```

```
END main
```

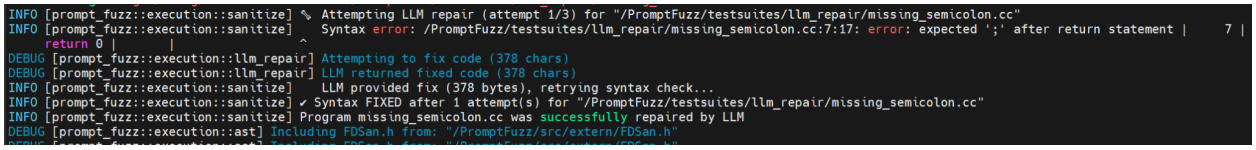
The finalized system operates in the following workflow:

Figure 2: LLM Repair Workflow



Here is a sample output of RePromptFuzz fixing a syntax error in a generated program:

Figure 3: Code Repair Output



Methodology

In order to properly test our design, we planned to run several tests between PromptFuzz and RePromptFuzz. Several factors needed to remain consistent throughout the tests due to the many different environment variables PromptFuzz allows users to configure. For our primary tests we used the following setup: Our AI model of choice for all testing was GPT 4.1 Nano 2025-4-14, chosen for its speed and relative poor coding abilities compared to other GPT versions. More errors would mean more chances for the repair code to trigger, presumably

increasing the measurable difference between PromptFuzz and RePromptFuzz. Additionally, the context limit was set to 8192, this was to account for a LLM issue that prevented contexts over a certain size from running. Our chosen library was Libpcap, for its relatively small size compared to the other libraries that PromptFuzz comes pre-compiled with. Lastly, all tests were performed on a 28 CPU core 256GB ram server with an NVIDIA Quadro RTX 5000 GPU. At some point we experimented with increasing the temperature of the LLM model to help induce errors, but this was later scrapped due to complications outlined in our findings.

Originally, our intention was to replicate Prompt Fuzz's original testing methodology as closely as possible, however numerous factors led us to vary our approach. The most blatant was the inability to use GPT 3.5, which the original PrompFuzz developers utilized in their tests. Similarly, we could not perfectly simulate the same hardware the researchers used. Therefore, we opted to re-do our own baseline tests.

After running our own baseline tests on PromptFuzz and compiling several metrics such as coverage rate, token / query use, and runtime, we would run several tests using RePromptFuzz, comparing the same metrics to determine if the iterative repair system had the same impact that was theorized.

Due to issues detailed in the section below, we were unable to perform our in-depth testing outlined above, however we were able to run several tests on the LLM repair module to ensure its functionality and get a rough estimate of its ability to fix code. This was done by running the module through several test cases that simulated syntactically incorrect output from the LLM and then attempted to repair it using the default 3 try limit. The eight cases we tried consisted of the following cases in C code:

- Valid Program

- Missing Braces
- Missing Semicolon
- Keyword Typo
- Missing Type
- Unmatched Parenthesis
- Multiple of the above

Successes and failures were logged to determine the effectiveness of our prompting and repair structure.

Findings

Unfortunately, we ran into numerous problems regarding PromptFuzz's original code that prevented us from completing our original intended testing. Upon setting up PromptFuzz and running our original test, we noticed that the LLM generated fuzz drivers had a zero percent error rate according to PromptFuzz's output. Originally, we concluded that this was a result of the improved quality of LLMs since the time of testing, it is quite possible that over the past year, enough improvements were made to get such a high level of accuracy. Still, we concluded that implementing RePromptFuzz would be of use for locally run LLM models. We set up a local LLM and a proxy address utilizing VLLM as per PromptFuzz's usage file. To our surprise, even the worst local LLM model we could get to run was still returning a 100% success rate. From there, we attempted to increase the temperature of both the local and paid LLMs to induce errors. Even at the highest possible temperature, we were unable to generate syntax errors or otherwise get our code to trigger. A bit of investigation revealed that at some point, PromptFuzz's error handling had been altered, meaning that the current public build of promptfuzz will never report

a syntax error correctly. We were able to sort through the code and implement a change to the JSON formatting of error data which restored what we presume was the original error reporting functionality of PromptFuzz. Unfortunately, the amount of time and effort involved in developing the fix for PromptFuzz's error detection in order to facilitate the testing of our code severely limited our testing period. When we finally began testing, we noticed that spontaneous errors were occurring during the fuzzing portion of PromptFuzz's execution. Given the time to run these tests was on average around 6 hours, with the error occurring randomly past the 30 minute mark, troubleshooting and fixing this issue was too time consuming for us to fix within our development time.

To account for these problems, we decided to dedicate our time to running the repair module tests. All eight test cases succeeded every time the tests were completed. The program was able to pass the valid program without attempting to make a repair and repair the following errors in a single attempt. Additionally, the program had no issues solving multiple problems in a single repair attempt.

Discussion

The repair module functioned as expected and was capable of completing various syntax repairs no matter the test that was performed. Additional cases also proved that the "looping" feature was also functioning correctly, as the program was able to solve problems on repeat tries.

From our troubleshooting and efforts to fix PromptFuzz, we noticed that many of the functions and strings within the program contain grammatical errors and other minor issues that made troubleshooting difficult and limited the usability of the program. We recommend that future work focuses on localizing the code and verifying its original output is sound before attempting to make any further changes. Once these groundwork issues have been fixed, we

assert that our contributions function and would urge further testing to test the hypothesis that iterative repair loops like RePromptFuzz improve performance of LLM powered fuzzing tools, or if PromptFuzz's original approach has more merit.

Limitations

Our work is not without some important clarifications. Due to the long period of time required for testing alongside the cost associated with using OpenAI's API, we were unable to perform a large number of trials for our improved version, this is regardless of the other errors we encountered when crafting out code. Switching to a local LLM model rectified the price concern, but increased the time required to generate responses, therefore not solving the time issue. Likewise, we could not test multiple libraries given our low testing count. Future iterations should consider the compute and financial resources available to determine what the most effective implementation of LLMs would be for testing within the allotted time span.

Conclusion

This paper introduces our concept for an iterative repair loop for PromptFuzz, dubbed RePromptFuzz. Unfortunately, due to numerous issues in the testing process caused by unforeseen errors in the original code, our hypothesis that such a repair loop might improve promptfuzz remains untested, although our proof of concept remains in a functional state, ready for future work to implement it. Depending on the findings of such work, the notion of utilizing repair loops in LLM power fuzzing tools may be supported, or PromptFuzz's original "brute force" approach may be revealed to be more advantageous.

References

Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. 2024. Prompt Fuzzing for Fuzz Driver Generation. In Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24).

Zhang, Cen & Mingqiang, Bai & Zheng, Yaowen & Li, Yeting & Xie, Xiaofei & Li, Yuekang & Ma, Wei & Sun, Limin & Liu, Yang. (2023). Understanding Large Language Model Based Fuzz Driver Generation. 10.48550/arXiv.2307.12469.

Amiet, N. (2023, December 7). Introducing fuzzomatic: Using AI to automatically fuzz rust projects from scratch. RSS. https://kudelskisecurity.com/research/introducing-fuzzomatic-using-ai-to-automatically-fuzz-rust-projects-from-scratch

Liu, D., M, J., & O, C. (2023). Fuzz target generation using llms. OSS-Fuzz. https://google.github.io/oss-fuzz/research/llms/target_generation/

Liu, D., M, J., & O, C. (2024). Leveling Up Fuzzing: Finding more vulnerabilities with AI. https://security.googleblog.com/2024/11/leveling-up-fuzzing-finding-more.html

Yu Jiang, Jie Liang, Fuchen Ma, Yuanliang Chen, Chijin Zhou, Yuheng Shen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Shanshan Li, and Quan Zhang. 2024. When Fuzzing Meets LLMs: Challenges and Opportunities. In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE 2024).